

# مستند فنی فرآیندهای ETL

## فهرست مطالب

|    |      |                                       |
|----|------|---------------------------------------|
| ۳  | ۱    | مقدمه                                 |
| ۳  | ۲    | استاندارد کلی مستندسازی               |
| ۴  | ۳    | ETL 1: Load_Fact_Lab_Event            |
| ۴  | ۱.۳  | هدف فرآیند                            |
| ۴  | ۲.۳  | مشخصات کلی                            |
| ۴  | ۳.۳  | جداول و داده‌های ورودی                |
| ۴  | ۴.۳  | منطق اجرای فرآیند                     |
| ۵  | ۵.۳  | نگاشت ستون‌ها                         |
| ۵  | ۶.۳  | کنترل اجرا و لاگ‌گیری                 |
| ۶  | ۷.۳  | مدیریت خطا                            |
| ۷  | ۴    | ETL 2: Load_Fact_Daily_ICU_Status     |
| ۷  | ۱.۴  | هدف فرآیند                            |
| ۷  | ۲.۴  | مشخصات کلی                            |
| ۷  | ۳.۴  | جداول موقت                            |
| ۷  | ۴.۴  | منطق بازه زمانی                       |
| ۸  | ۵.۴  | منطق اجرای فرآیند                     |
| ۸  | ۶.۴  | منابع داده و تجمیع                    |
| ۹  | ۷.۴  | منطق درج در جدول مقصد                 |
| ۹  | ۸.۴  | نگاشت ستون‌های اصلی                   |
| ۱۰ | ۹.۴  | کنترل اجرا و لاگ‌گیری                 |
| ۱۰ | ۱۰.۴ | مدیریت خطا                            |
| ۱۱ | ۵    | ETL 3: Load_Fact_ICU_Clinical_Journey |

|    |                        |      |
|----|------------------------|------|
| ۱۱ | هدف فرآیند             | ۱.۵  |
| ۱۱ | مشخصات کلی             | ۲.۵  |
| ۱۱ | جدول واسط              | ۳.۵  |
| ۱۱ | منطق بارگذاری          | ۴.۵  |
| ۱۱ | منطق اجرای فرآیند      | ۵.۵  |
| ۱۲ | ساخت دیتاست میانی      | ۶.۵  |
| ۱۲ | شاخص‌های محاسباتی      | ۷.۵  |
| ۱۲ | فلگ‌های منطقی          | ۸.۵  |
| ۱۳ | نگاشت ستون‌ها          | ۹.۵  |
| ۱۳ | منطق به‌روزرسانی و درج | ۱۰.۵ |
| ۱۳ | کنترل اجرا و لاگ‌گیری  | ۱۱.۵ |
| ۱۴ | مدیریت خطا             | ۱۲.۵ |

## ۱ مقدمه

این سند، مستند فنی سه فرآیند ETL در لایه انبار داده را ارائه می‌کند. هدف این مستند، تشریح منطق بارگذاری، منابع داده، جداول واسط، نحوه کنترل اجرا، مدیریت خطا و ساختار داده‌های خروجی برای هر ETL است.

فرآیندهای مستندسازی شده در این سند شامل موارد زیر هستند:

- Load\_Fact\_Lab\_Event
- Load\_Fact\_Daily\_ICU\_Status
- Load\_Fact\_ICU\_Clinical\_Journey

## ۲ استاندارد کلی مستندسازی

برای هر ETL، بخش‌های زیر ارائه شده‌اند:

۱. هدف فرآیند
۲. نام رویه و نوع بارگذاری
۳. جداول مبدأ، مقصد و واسط
۴. منطق اجرایی
۵. نگاشت ستون‌ها
۶. کنترل اجرا و لاگ‌گیری
۷. مدیریت خطا
۸. پیش‌نیازهای اجرا

## ۳ ETL 1: Load\_Fact\_Lab\_Event

### ۱.۳ هدف فرآیند

این ETL برای بارگذاری داده‌های رویدادهای آزمایشگاهی از لایه Staging به جدول واقعیت Fact\_Lab\_Event طراحی شده است. بارگذاری به صورت افزایشی انجام می‌شود و مبنای تشخیص رکوردهای جدید، ستون ROW\_ID در جدول مبدأ است.

### ۲.۳ مشخصات کلی

| مشخصه        | مقدار                                            |
|--------------|--------------------------------------------------|
| نام رویه     | dbo.Load_Fact_Lab_Event                          |
| نوع بارگذاری | Incremental Load                                 |
| جدول مبدأ    | DW_Staging.Stage.LAB_EVENTS                      |
| جداول بُعد   | dbo.Dim_Patient, dbo.Dim_Date, dbo.Dim_Lab_Items |
| جدول مقصد    | dbo.Fact_Lab_Event                               |
| جدول کنترل   | dbo.ETL_Control                                  |
| رویه ثبت لاگ | dbo.sp_Insert_ETL_Log                            |

### ۳.۳ جداول و داده‌های ورودی

ستون‌های اصلی مورد استفاده از جدول LAB\_EVENTS عبارت‌اند از:

- ROW\_ID: شناسه یکتا برای ردیابی بارگذاری
- CHART\_TIME: زمان ثبت رویداد آزمایش
- PATIENT\_ID: شناسه بیمار
- ITEM\_ID: شناسه آیتم آزمایش
- ADMISSION\_ID: شناسه بستری
- VALUE\_NUM: مقدار عددی آزمایش
- VALUE: مقدار متنی آزمایش
- VALUE\_UOM: واحد اندازه‌گیری
- FLAG: وضعیت نتیجه

### ۴.۳ منطق اجرای فرآیند

منطق اجرایی این ETL به ترتیب زیر است:

۱. بررسی می‌شود که آیا برای LAB\_EVENTS در جدول ETL\_Control رکورد کنترلی وجود دارد یا خیر.

۲. اگر رکورد کنترلی وجود نداشته باشد، یک رکورد اولیه با مقدار  $Last\_Processed\_ID = 0$  و وضعیت INITIAL درج می‌شود.
۳. آخرین مقدار پردازش شده از ستون  $Last\_Processed\_ID$  خوانده می‌شود.
۴. از جدول LAB\_EVENTS فقط رکوردهایی انتخاب می‌شوند که ROW\_ID آن‌ها از مقدار ذخیره شده بیشتر باشد.
۵. داده‌ها با جدول Dim\_Date براساس تاریخ CHART\_TIME، با جدول Dim\_Patient براساس PATIENT\_ID و با جدول Dim\_Lab\_Items براساس ITEM\_ID اتصال داده می‌شوند.
۶. خروجی حاصل در جدول Fact\_Lab\_Event درج می‌شود.
۷. بلافاصله بعد از درج، تعداد رکوردهای درج شده با @@ROWCOUNT ذخیره می‌شود.
۸. سپس بیشترین ROW\_ID جدید از جدول مبدأ استخراج می‌شود.
۹. جدول ETL\_Control با وضعیت SUCCESS، زمان اجرا و مقدار جدید  $Last\_Processed\_ID$  به روزرسانی می‌شود.
۱۰. در پایان، نتیجه اجرا در لاگ ETL ثبت می‌شود.

### ۵.۳ نگاشت ستون‌ها

| منبع                                                      | ستون مقصد    |
|-----------------------------------------------------------|--------------|
| از Dim_Date براساس CHART_TIME<br>CAST(CHART_TIME AS DATE) | Date_SK      |
| از Dim_Patient براساس PATIENT_ID                          | Patient_SK   |
| از Dim_Lab_Items براساس ITEM_ID                           | LabTest_SK   |
| s.ADMISSION_ID                                            | Admission_ID |
| s.VALUE_NUM                                               | Value_Num    |
| s.VALUE                                                   | Value_Text   |
| s.VALUE_UOM                                               | Value_Unit   |
| s.FLAG                                                    | Flag         |

### ۶.۳ کنترل اجرا و لاگ‌گیری

در این فرآیند از جدول ETL\_Control برای ذخیره اطلاعات کنترلی زیر استفاده می‌شود:

- Table\_Name = 'LAB\_EVENTS'
- Last\_Processed\_ID
- Last\_Run\_Status
- Last\_Run\_Timestamp
- Error\_Message

همچنین از رویه `sp_Insert_ETL_Log` برای ثبت اطلاعات اجرای ETL شامل نام رویه، نوع عملیات، نام جدول مقصد و تعداد رکوردهای متاثر استفاده می‌شود.

### ۷.۳ مدیریت خطا

این ETL از ساختار `TRY...CATCH` استفاده می‌کند. در صورت بروز خطا:

- پیام خطا با `ERROR_MESSAGE()` دریافت می‌شود.
- وضعیت اجرا در جدول `ETL_Control` برابر `FAILED` ثبت می‌شود.
- زمان اجرای ناموفق ثبت می‌شود.
- متن خطا در ستون `Error_Message` ذخیره می‌شود.
- یک رکورد لاگ با تعداد ردیف صفر درج می‌شود.
- خطا با `THROW` مجدداً ارسال می‌شود.

## ETL 2: Load\_Fact\_Daily\_ICU\_Status ۴

### ۱.۴ هدف فرآیند

این ETL برای تولید وضعیت روزانه بیماران بستری در ICU طراحی شده است. داده‌های ورودی مربوط به جریان‌های ورودی و خروجی مایعات، ابتدا در جداول موقت تجمع می‌شوند و سپس در جدول Fact\_Daily\_ICU\_Status بارگذاری می‌گردند.

### ۲.۴ مشخصات کلی

| مشخصه        | مقدار                                                                                        |
|--------------|----------------------------------------------------------------------------------------------|
| نام رویه     | dbo.Load_Fact_Daily_ICU_Status                                                               |
| نوع بارگذاری | Incremental / Sliding Window Reload                                                          |
| جدول مبدأ    | ،ICU_INPUT_EVENTS_CV<br>،ICU_INPUT_EVENTS_MV<br>،ICU_D_ITEMS ،ICU_OUTPUT_EVENTS<br>ICU_STAYS |
| جدول واسط    | ،dbo.Tmp_OutputData ،dbo.Tmp_InputData<br>،dbo.Tmp_Aggregated                                |
| جدول بُعد    | dbo.Dim_Patient ،dbo.Dim_Date                                                                |
| جدول مقصد    | dbo.Fact_Daily_ICU_Status                                                                    |
| جدول کنترل   | dbo.ETL_Control                                                                              |
| رویه ثبت لاگ | dbo.sp_Insert_ETL_Log                                                                        |

### ۳.۴ جداول موقت

در این ETL سه جدول موقت دائمی در اسکیمای dbo استفاده شده‌اند:

۱. Tmp\_InputData: ذخیره داده‌های ورودی مایعات و خون
۲. Tmp\_OutputData: ذخیره داده‌های خروجی مانند ادرار و درناژ
۳. Tmp\_Aggregated: نگهداری مقادیر نهایی تجمع‌شده در سطح بیمار، بستری ICU و تاریخ

### ۴.۴ منطق بازه زمانی

این ETL براساس تاریخ عمل می‌کند و از یک پنجره بازبارگذاری استفاده می‌کند:

- Last\_Processed\_Date از جدول ETL\_Control خوانده می‌شود.
- MaxSafeDate برابر روز قبل از تاریخ جاری در نظر گرفته می‌شود.
- ReloadStartDate برابر دو روز قبل از آخرین تاریخ پردازش شده است.
- اگر مقدار قبلی وجود نداشته باشد، تاریخ شروع برابر 1900-01-01 قرار می‌گیرد.

## ۵.۴ منطق اجرای فرآیند

مراحل اجرای این ETL به شرح زیر است:

۱. بررسی و ایجاد رکورد کنترلی برای ICU\_DAILY\_STATUS
۲. خواندن Last\_Processed\_Date
۳. محاسبه ReloadStartDate و MaxSafeDate
۴. پاکسازی جداول Tmp\_InputData، Tmp\_OutputData و Tmp\_Aggregated با TRUNCATE TABLE
۵. بارگذاری داده‌های ورودی از ICU\_INPUT\_EVENTS\_CV و ICU\_INPUT\_EVENTS\_MV در Tmp\_InputData
۶. بارگذاری داده‌های خروجی از ICU\_OUTPUT\_EVENTS در Tmp\_OutputData
۷. تجمیع داده‌های ورودی و خروجی در Tmp\_Aggregated با استفاده از FULL OUTER JOIN
۸. حذف داده‌های قبلی جدول Fact\_Daily\_ICU\_Status برای بازه بازبارگذاری
۹. درج مجدد داده‌های بازه مورد نظر در جدول مقصد
۱۰. محاسبه تعداد رکوردهای درج شده
۱۱. استخراج آخرین تاریخ واقعی پردازش شده از جدول Fact
۱۲. به روزرسانی جدول کنترل و ثبت لاگ

## ۶.۴ منابع داده و تجمیع

منابع داده در این ETL به شکل زیر استفاده می‌شوند:

- ICU\_INPUT\_EVENTS\_CV: دریافت رخداد‌های ورودی از نوع CareVue
- ICU\_INPUT\_EVENTS\_MV: دریافت رخداد‌های ورودی از نوع MetaVision
- ICU\_OUTPUT\_EVENTS: دریافت خروجی‌های بیمار
- ICU\_D\_ITEMS: تعیین دسته‌بندی هر ITEM\_ID

تجمیع داده‌ها در جدول Tmp\_Aggregated بر اساس ستون‌های زیر انجام می‌شود:

• SUBJECT\_ID

• ICU\_STAY\_ID

• SnapshotDate

مقادیر تجمیعی شامل این فیلدها هستند:



- Fluid\_Input\_ml: مجموع مقادیر دسته‌هایی که شامل Intake هستند
- Blood\_Input\_ml: مجموع مقادیر دسته‌هایی که شامل Blood هستند
- Urine\_Output\_ml: مجموع مقادیر دسته‌هایی که شامل Urine هستند
- Drain\_Output\_ml: مجموع مقادیر دسته‌هایی که شامل Drain هستند

#### ۷.۴ منطق درج در جدول مقصد

در زمان درج در Fact\_Daily\_ICU\_Status، فیلدهای زیر تولید می‌شوند:

- کلید تاریخ از Dim\_Date
  - کلید بیمار از Dim\_Patient
  - مقادیر ورودی و خروجی تجمیع شده
  - مجموع کل ورودی‌ها
  - مجموع کل خروجی‌ها
  - تراز خالص مایع
  - شماره روز بستری ICU بر اساس اختلاف بین IN\_TIME و SnapshotDate
- اگر تاریخ Snapshot خارج از بازه بستری ICU باشد، مقدار روز بستری برابر NULL در نظر گرفته می‌شود.

#### ۸.۴ نگاشت ستون‌های اصلی

| منبع / منطق                          | ستون مقصد         |
|--------------------------------------|-------------------|
| از Dim_Date بر اساس SnapshotDate     | Date_SK           |
| از Dim_Patient بر اساس SUBJECT_ID    | Patient_SK        |
| از ICU_STAY_ID                       | ICU_STAY_ID       |
| از Tmp_Aggregated                    | Fluid_Input_ml    |
| از Tmp_Aggregated                    | Blood_Input_ml    |
| از Tmp_Aggregated                    | Urine_Output_ml   |
| از Tmp_Aggregated                    | Drain_Output_ml   |
| Fluid_Input_ml + Blood_Input_ml      | Total_Input       |
| Urine_Output_ml + Drain_Output_ml    | Total_Output      |
| Total_Input - Total_Output           | Net_Fluid_Balance |
| DATEDIFF(DAY, IN_TIME, SnapshotDate) | ICU_Day_Number    |

## ۹.۴ کنترل اجرا و لاگ‌گیری

اطلاعات کنترلی این ETL با مقدار `Table_Name = 'ICU_DAILY_STATUS'` در جدول `ETL_Control` ذخیره می‌شود. فیلدهای اصلی مورد استفاده عبارت‌اند از:

- `Last_Processed_Date`
- `Last_Run_Status`
- `Last_Run_Timestamp`
- `Error_Message`

پس از پایان اجرای موفق، مقدار آخرین تاریخ پردازش شده از خود جدول `Fact` و جدول `Dim_Date` به دست می‌آید و در جدول کنترل ذخیره می‌شود.

## ۱۰.۴ مدیریت خطا

در صورت بروز خطا:

- پیام خطا دریافت می‌شود.
- وضعیت اجرا در `ETL_Control` برابر `FAILED` ثبت می‌گردد.
- زمان بروز خطا ثبت می‌شود.
- متن خطا در ستون `Error_Message` ذخیره می‌شود.
- یک رکورد لاگ با تعداد ردیف صفر درج می‌شود.
- خطا با `THROW` به فراخوان بالاتر ارسال می‌شود.

## ETL 3: Load\_Fact\_ICU\_Clinical\_Journey ۵

### ۱.۵ هدف فرآیند

این ETL برای ساخت و به روز رسانی مسیر بالینی بیمار در ICU طراحی شده است. داده‌های مربوط به پذیرش ICU، تهویه مکانیکی، آنتی بیوتیک، دیالیز، مدت بستری و وضعیت فوت بیمار در یک جدول واقعیت به نام Fact\_ICU\_Clinical\_Journey ذخیره می‌شوند.

### ۲.۵ مشخصات کلی

| مشخصه        | مقدار                                                                                   |
|--------------|-----------------------------------------------------------------------------------------|
| نام رویه     | dbo.Load_Fact_ICU_Clinical_Journey                                                      |
| نوع بارگذاری | Incremental Load with Update/Insert                                                     |
| جدول مبدأ    | ICU_STAYS, Clinic_ADMISSIONS, ICU_PROCEDURE_EVENTS_MV, ICU_D_ITEMS, ICU_INPUT_EVENTS_MV |
| جدول واسط    | dbo.Tmp_ICU_Journey                                                                     |
| جدول بُعد    | dbo.Dim_Date, dbo.Dim_Patient                                                           |
| جدول مقصد    | dbo.Fact_ICU_Clinical_Journey                                                           |
| جدول کنترل   | dbo.ETL_Control                                                                         |
| رویه ثبت لاگ | dbo.sp_Insert_ETL_Log                                                                   |

### ۳.۵ جدول واسط

جدول Tmp\_ICU\_Journey برای نگهداری مجموعه داده نهایی قبل از UPDATE و INSERT در جدول Fact استفاده می‌شود. این جدول شامل اطلاعات زمانی، شاخص‌های عددی، فلگ‌ها و خروجی نهایی بیمار است.

### ۴.۵ منطق بارگذاری

این ETL بر مبنای زمان آخرین تغییر اجرا می‌شود. مقدار Last\_Modified\_Timestamp از جدول ETL\_Control خوانده می‌شود و فقط ICU های stay انتخاب می‌شوند که زمان شروع یا پایان آن‌ها از آخرین اجرای ثبت شده جدیدتر باشد.

### ۵.۵ منطق اجرای فرآیند

مراحل اصلی ETL به شرح زیر هستند:

۱. بررسی وجود رکورد کنترلی برای Fact\_ICU\_Clinical\_Journey
۲. در صورت نبود رکورد، ایجاد مقدار اولیه در جدول ETL\_Control
۳. خواندن Last\_Modified\_Timestamp و قرار دادن مقدار پیش فرض 1900-01-01 در صورت NULL
۴. پاک‌سازی جدول Tmp\_ICU\_Journey

۵. ساخت مجموعه داده میانی با استفاده از اطلاعات ICU، stay پذیرش بیمارستان، تهویه، آنتی بیوتیک و دیالیز

۶. به روزرسانی رکوردهای موجود در Fact\_ICU\_Clinical\_Journey براساس ICU\_Stay\_ID

۷. درج رکوردهای جدیدی که قبلاً در جدول مقصد وجود نداشته اند

۸. محاسبه تعداد رکوردهای به روزرسانی شده و درج شده

۹. به روزرسانی وضعیت اجرا در ETL\_Control

۱۰. ثبت نتیجه اجرا در جدول لاگ

## ۶.۵ ساخت دیتاست میانی

دیتاست میانی از چند بخش تشکیل می شود:

۱. بخش پایه: از جدول ICU\_STAYS و Clinic\_ADMISSIONS اطلاعات اصلی شامل شناسه بیمار، شناسه بستری، زمان ورود و خروج ICU و فلگ فوت بیمارستانی استخراج می شود.

۲. بخش تهویه مکانیکی: از جدول ICU\_PROCEDURE\_EVENTS\_MV و ICU\_D\_ITEMS اولین زمان شروع و آخرین زمان پایان رویدادهای مرتبط با دسته Respiratory استخراج می شود.

۳. بخش آنتی بیوتیک: از جدول ICU\_INPUT\_EVENTS\_MV و ICU\_D\_ITEMS اولین زمان دریافت آیتمی که برچسب آن ها شامل antibiotic است استخراج می شود.

۴. بخش دیالیز: از جدول ICU\_PROCEDURE\_EVENTS\_MV و ICU\_D\_ITEMS اولین زمان رویدادهایی که برچسب آن ها شامل dialysis است استخراج می شود.

## ۷.۵ شاخص های محاسباتی

در جدول Tmp\_ICU\_Journey شاخص های زیر محاسبه می شوند:

- Time\_To\_Vent\_Hours: فاصله زمانی بین پذیرش ICU و شروع تهویه
- Vent\_Duration\_Hours: فاصله زمانی بین شروع و پایان تهویه
- Time\_To\_Antibiotic\_Hours: فاصله زمانی بین پذیرش ICU و اولین آنتی بیوتیک
- LOS\_ICU\_Hours: مدت بستری ICU

## ۸.۵ فلگ های منطقی

فلگ های زیر در این ETL تولید می شوند:

- Ventilated\_Flag: در صورت وجود اطلاعات تهویه برابر ۱
- Sepsis\_Suspected\_Flag: در صورت وجود زمان اولین آنتی بیوتیک برابر ۱
- Dialysis\_Flag: در صورت وجود زمان شروع دیالیز برابر ۱
- Mortality\_Flag: برابر مقدار HOSPITAL\_EXPIRE\_FLAG

## ۹.۵ نگاشت ستون‌ها

| منبع / منطق                                    | ستون مقصد                |
|------------------------------------------------|--------------------------|
| b.ICU_STAY_ID                                  | ICU_Stay_ID              |
| SUBJECT_ID از Dim_Patient براساس               | Patient_SK               |
| IN_TIME از Dim_Date براساس تاریخ               | Admit_Date_SK            |
| b.IN_TIME                                      | ICU_Admit_Time           |
| از زیرپرس وجوی تهویه                           | Vent_Start_Time          |
| از زیرپرس وجوی تهویه                           | Vent_End_Time            |
| از زیرپرس وجوی آنتی‌بیوتیک                     | First_Antibiotic_Time    |
| از زیرپرس وجوی دیالیز                          | Dialysis_Start_Time      |
| b.OUT_TIME                                     | ICU_Discharge_Time       |
| DATEDIFF(HOUR, IN_TIME, Vent_Start_Time)       | Time_To_Vent_Hours       |
| DATEDIFF(HOUR, Vent_Start_Time, Vent_End_Time) | Vent_Duration_Hours      |
| DATEDIFF(HOUR, IN_TIME, First_Antibiotic_Time) | Time_To_Antibiotic_Hours |
| DATEDIFF(HOUR, IN_TIME, OUT_TIME)              | LOS_ICU_Hours            |
| وجود رکورد تهویه                               | Ventilated_Flag          |
| وجود رکورد آنتی‌بیوتیک                         | Sepsis_Suspected_Flag    |
| وجود رکورد دیالیز                              | Dialysis_Flag            |
| HOSPITAL_EXPIRE_FLAG                           | Mortality_Flag           |

## ۱۰.۵ منطق به‌روزرسانی و درج

این ETL دو مرحله اصلی بر روی جدول مقصد انجام می‌دهد:

۱. UPDATE: اگر ICU\_Stay\_ID از قبل در جدول مقصد موجود باشد، ستون‌های زمانی، مقادیر محاسباتی و فلگ‌ها به‌روزرسانی می‌شوند.

۲. INSERT: اگر ICU\_Stay\_ID در جدول مقصد وجود نداشته باشد، رکورد جدید درج می‌شود.

## ۱۱.۵ کنترل اجرا و لاگ‌گیری

برای این ETL از رکوردی با مقدار 'Fact\_ICU\_Clinical\_Journey' در جدول ETL\_Control استفاده می‌شود. فیلدهای اصلی عبارت‌اند از:

- Last\_Run\_Status
- Last\_Run\_Timestamp
- Last\_Modified\_Timestamp
- Error\_Message

در پایان اجرای موفق، زمان جاری در Last\_Run\_Timestamp و Last\_Modified\_Timestamp ثبت می‌شود.

## ۱۲.۵ مدیریت خطا

در صورت بروز خطا در این ETL:

- وضعیت اجرا در جدول کنترل برابر FAILED ثبت می شود.
- پیام خطا در ستون Error\_Message ذخیره می گردد.
- زمان اجرای ناموفق ثبت می شود.
- یک رکورد لاگ با تعداد ردیف های درج شده تا لحظه بروز خطا ثبت می شود.